SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Industry Problem Solving Task (Medium)
Assessment Tool Weightage: 35%

Dr

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	**5**
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	**5**
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	**5**
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	**5**
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	**5**
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	**5**
7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	**5**
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	**5**

9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	**5**
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	**5**

Code of Conduct:

*I **RAVURU PRANATHI (192525076)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: R. Pranathi

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution

		standards			
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.

File Organization: Indexed Sequential File Organization

Each product record can contain:

- Product_ID – Primary key
- Product_Name
- Category
- Price
- Stock
- Seller_ID

The records should be stored on disk in **data blocks**, with a **B+ Tree index on Product_ID**.

Why this is suitable

1. **Fast search**
    - B+ Tree supports efficient searching using Product_ID.
    - Search cost is approximately **O(log N)** instead of scanning all 10 million records.
2. **Efficient range queries**
    - Queries such as:
        - Products with IDs 10000–20000
        - Products priced between ₹500 and ₹1000

    - can be handled efficiently using the B+ Tree.
3. **Good insertion and deletion**
    - E-commerce databases frequently add and remove products.
    - B+ Trees dynamically maintain the index without reorganizing the entire file.
4. **Good scalability**
    - The structure can handle millions of records and grow as the number of products increases.

Cost Analysis

Assume:

- Number of records = **10,000,000**
- Average record size = **200 bytes**
- Block size = **4 KB = 4096 bytes**

Approximate records per block:

$$\frac{4096}{200} \approx 20$$

Number of data blocks:

$$\frac{10,000,000}{20} = 500,000 \text{ blocks}$$

Sequential File

Searching for a product may require scanning up to:

500,000 I/O operations

Average search:

$$\frac{500,000}{2} = 250,000 \text{ I/Os}$$

This is very expensive for frequently accessed e-commerce data.

B+ Tree Indexed File

A B+ Tree with a reasonable fan-out might require only around **3–5 index levels**.

Therefore, a product lookup may require approximately:

$3-5$ I/O operations

So, approximately:

Method	Worst-case I/O	Average/Typical Search
Sequential file	500,000	250,000
B+ Tree indexed file	3–5	3–5

Conclusion

For **10 million product records**, an **indexed sequential file organization using a B+ Tree index** is the most appropriate choice. Although maintaining the index requires additional storage and update cost, the dramatic reduction in disk I/O makes it much more efficient for an e-commerce system where product searches, updates, and range queries occur frequently.

2. A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.

Indexing Strategy for Banking Transactions

For a banking system, transactions must be retrieved quickly using **account number** and **date range**. A **composite B+ Tree index** is the most suitable choice.

1. Transaction Table

Example:

```
TRANSACTION(
   Transaction_ID,
   Account_No,
   Transaction_Date,
   Transaction_Type,
   Amount,
   Balance
)
```

2. Primary Index

Create a **B+ Tree index** on Transaction_ID.

```
CREATE INDEX idx_transaction_id
ON TRANSACTION(Transaction_ID);
```

This provides fast access to an individual transaction.

3. Composite Index

Create a composite B+ Tree index on:

```
(Account_No, Transaction_Date)
CREATE INDEX idx_account_date
ON TRANSACTION(Account_No, Transaction_Date);
```

This is the **most important index** for the requirement.

For example:

```
SELECT *
FROM TRANSACTION
WHERE Account_No = 10025
AND Transaction_Date BETWEEN '2026-01-01' AND '2026-03-31';
```

The database can first locate the required Account_No and then efficiently scan only the required **date range**.

4. Why B+ Tree?

B+ Tree is appropriate because:

- Supports fast equality searches on account number.
- Supports efficient **range searches** on dates.
- Keeps keys sorted.
- Leaf nodes are linked, making sequential range retrieval efficient.
- Works well for very large transaction tables.

5. Optional Index

If the bank frequently searches transactions **only by date**, create another B+ Tree index:

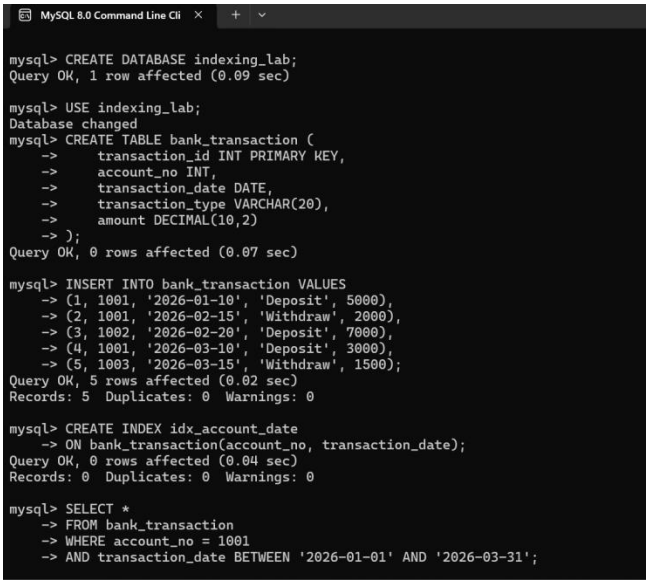
```
CREATE INDEX idx_transaction_date
ON TRANSACTION(Transaction_Date);
```

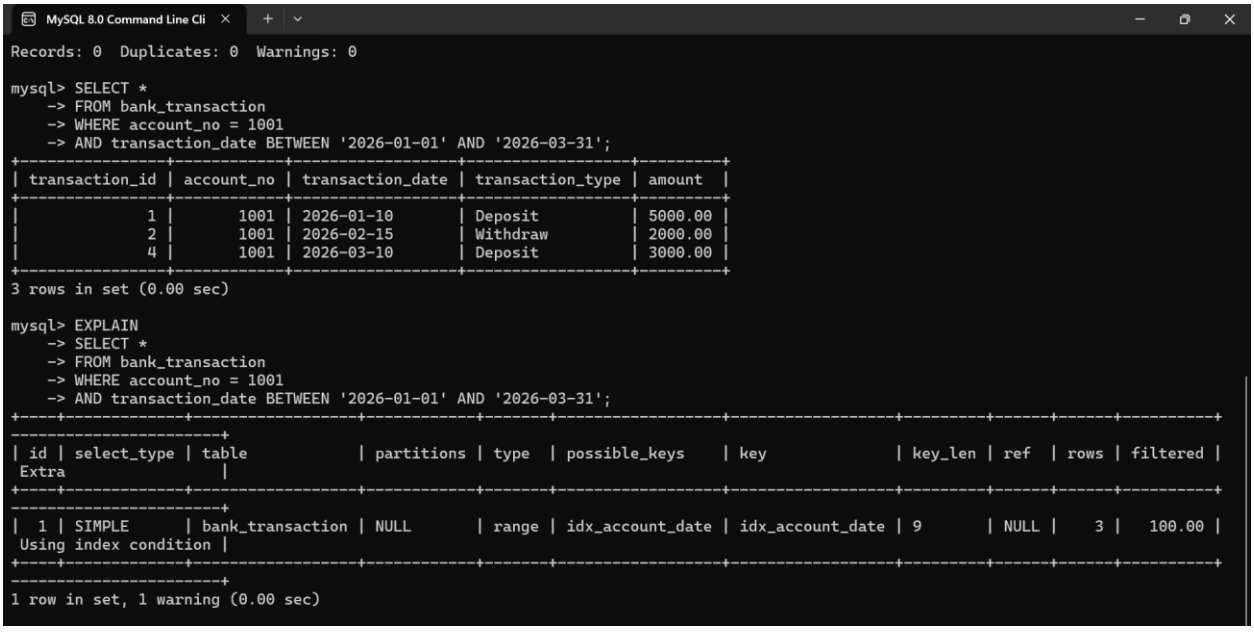
However, this should be created only when such queries are common because every additional index requires extra storage and increases insertion/update cost.

Recommended Design

Query Requirement	Index Type	Indexed Columns
Find transaction by ID	B+ Tree	Transaction_ID
Find transactions for an account	B+ Tree	Account_No
Account + date range	**Composite B+ Tree**	**Account_No, Transaction_Date**
Date-only searches	B+ Tree	Transaction_Date

Conclusion: The best primary strategy is a **composite B+ Tree index on (Account_No, Transaction_Date)**. It provides fast account-based retrieval while also making date-range queries efficient, which is ideal for banking transaction systems.





3. A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.

Multi-Level Indexing Solution for Healthcare System

A healthcare system needs to retrieve patient records using **PatientID** and **Doctor Name**. Since the database may contain millions of records, a **multi-level indexing strategy using B+ Trees** is suitable.

1. Patient Table

```
PATIENT(
   PatientID,
   PatientName,
   Age,
   Gender,
   DoctorName,
   Diagnosis,
   AdmissionDate
)
```

2. Primary Index – PatientID

Create a **B+ Tree index** on PatientID.

```
CREATE INDEX idx_patient_id
ON PATIENT(PatientID);
```

Purpose: Quickly locate a specific patient's record.

Example:

```
SELECT *
FROM PATIENT
WHERE PatientID = 10520;
```

The B+ Tree avoids scanning the entire patient table.

3. Secondary Index – DoctorName

Create another **B+ Tree secondary index** on DoctorName.

```
CREATE INDEX idx_doctor_name
ON PATIENT(DoctorName);
```

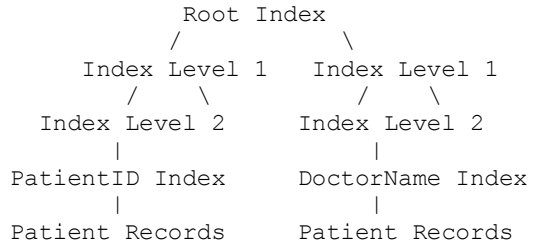
Purpose: Retrieve all patients assigned to a particular doctor.

Example:

```
SELECT *
FROM PATIENT
WHERE DoctorName = 'Dr. Kumar';
```

4. Multi-Level Structure

The indexing structure can be represented as:



Each index level reduces the number of disk blocks that must be searched.

5. Why B+ Tree?

B+ Tree is appropriate because it:

- Provides fast searching with **O(log N)** complexity.
- Supports large numbers of patient records.
- Reduces disk I/O.
- Supports equality searches efficiently.
- Can also support range queries such as PatientID ranges.
- Keeps index entries sorted.

6. Cost Consideration

Suppose the healthcare system has **1 million patient records**.

Without an index, searching for a patient could require scanning many records.

With a multi-level B+ Tree, the search may require only a few disk accesses:

Root → Intermediate Node → Leaf Node → Patient Record

So the approximate search cost is:

O(logBN)

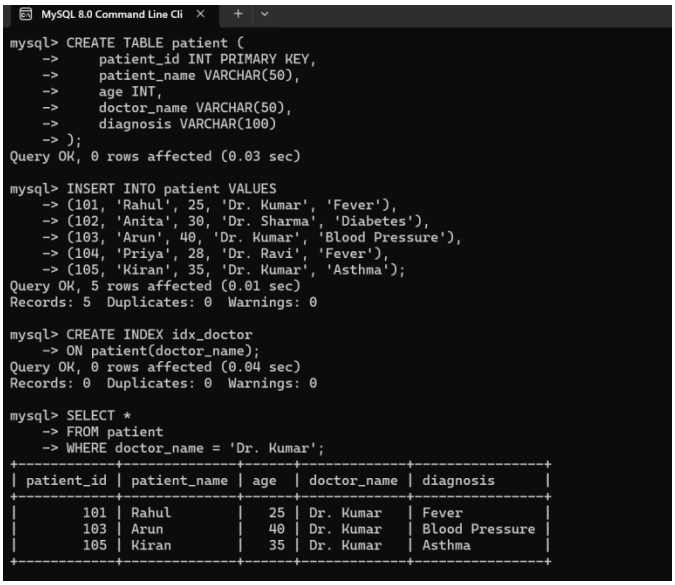
where **B** is the number of entries that can fit in an index node.

Final Design

Access Requirement	Index	Type
Search by PatientID	PatientID	Primary B+ Tree
Search by Doctor Name	DoctorName	Secondary B+ Tree
Large-scale indexing	Multi-level B+ Tree	Hierarchical
Range search	PatientID B+ Tree	Efficient

Conclusion:

Use a **multi-level B+ Tree indexing system**, with a primary index on PatientID and a secondary index on DoctorName. This provides fast and scalable retrieval regardless of whether users search by a specific patient or by the doctor responsible for multiple patients.



```
mysql> EXPLAIN
    -> SELECT *
    -> FROM patient
    -> WHERE doctor_name = 'Dr. Kumar';
+----+-------------+---------+------------+------+---------------+------------+---------+-------+------+----------+-------+
| id | select_type | table   | partitions | type | possible_keys | key        | key_len | ref   | rows | filtered | Extra |
+----+-------------+---------+------------+------+---------------+------------+---------+-------+------+----------+-------+
|  1 | SIMPLE      | patient | NULL       | ref  | idx_doctor    | idx_doctor | 203     | const |    3 |   100.00 | NULL  |
+----+-------------+---------+------------+------+---------------+------------+---------+-------+------+----------+-------+
1 row in set, 1 warning (0.00 sec)
```

4. A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.

B-Tree vs B+ Tree for Logistics Shipment Records

A logistics company continuously **inserts, updates, and deletes shipment records**. Therefore, the indexing structure should support dynamic operations efficiently while also allowing fast searches and range queries.

Comparison

Feature	**B-Tree**	**B+ Tree**
Data stored	Internal + leaf nodes	Only leaf nodes
Search	Fast	Fast
Insertion	Efficient	Efficient
Deletion	Efficient	Efficient
Range queries	Less efficient	**Very efficient**
Sequential access	Moderate	**Excellent**
Fan-out	Lower	**Higher**
Disk I/O	More	**Less**
Suitable for databases	Good	**Excellent**

Why B+ Tree is Better

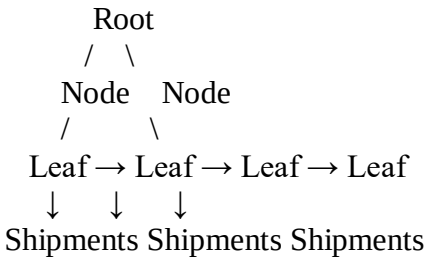
A **B+ Tree** is more suitable for the logistics system because shipment records are often accessed using ranges.

For example:

SELECT *
FROM Shipment

WHERE Shipment_Date BETWEEN '2026-08-01' AND '2026-08-25';

The B+ Tree stores all actual data entries in its leaf nodes, and the leaf nodes are linked:



Once the first matching shipment is found, the linked leaf nodes allow the system to retrieve subsequent records efficiently.

Dynamic Insertions

When a new shipment is added, the B+ Tree inserts the key into the appropriate leaf node.

If the node becomes full:

Leaf Overflow
↓
Split Node
↓
Promote Key to Parent

The tree remains balanced.

Dynamic Deletions

When a shipment is deleted, the B+ Tree removes the corresponding entry.

If a node becomes too empty, it can:

1. Borrow an entry from a neighboring node, or
2. Merge with a neighboring node.

Thus, the tree remains balanced after deletions.

Final Recommendation

B+ Tree is more suitable for the logistics company.

The main reasons are:

- Efficient dynamic insertion and deletion.
- Lower disk I/O.
- High fan-out and smaller tree height.
- Excellent sequential access.
- Very efficient range queries.
- Maintains balanced structure as shipment records grow or shrink.

Conclusion:

For a database-oriented logistics system with frequent shipment insertions, deletions, searches, and date/status range queries, **B+ Tree should be preferred over B-Tree**.

5. Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.

Hashing Scheme for HR System with 5000 Employees

An HR system can use **EmployeeID** as the search key because it is unique for every employee.

1. Proposed Hash Function

Suppose there are **100 buckets** initially.

Use:

h(EmployeeID)=EmployeeIDmod100

For example:

EmployeeID	Hash Calculation	Bucket
1025	1025 % 100	25
2137	2137 % 100	37
4502	4502 % 100	2
9876	9876 % 100	76

Each bucket stores employee records having the same hash value.

2. Static Hashing

In **static hashing**, the number of buckets remains fixed.

For 5000 employees:

1005000=50

So, approximately **50 employees per bucket**.

When a bucket becomes full, **overflow buckets** are required.

EmployeeID
↓
Hash Function
↓
Bucket 0 ⟶ Overflow
Bucket 1
Bucket 2 ⟶ Overflow
...
Bucket 99

Advantages:

- Simple to implement.
- Fast direct access.
- Low implementation complexity.

Disadvantages:

- Employee numbers may increase in the future.
- Overflow chains can become long.
- Performance decreases when many overflow records occur.
- Resizing the entire hash structure is expensive.

3. Extendible Hashing

For an HR system, **extendible hashing** is more flexible.

It uses:

- A **directory**
- Multiple buckets
- **Global depth**
- **Local depth**

Initially, suppose:

Global Depth=2

The directory has:

22=4

entries.

Directory

00 → Bucket A
01 → Bucket B
10 → Bucket C
11 → Bucket D

When a bucket overflows, it is split. If necessary, the directory is doubled.

Before:

00 → A
01 → B
10 → C
11 → D

Bucket B overflow
↓
Directory doubles

000 → A
001 → A
010 → B1
011 → B2
100 → C
101 → C
110 → D
111 → D

Only the affected bucket needs to be reorganized.

4. Static vs Extendible Hashing

Feature	**Static Hashing**	**Extendible Hashing**
Number of buckets	Fixed	Dynamic
Overflow handling	Overflow buckets	Bucket splitting
Growth handling	Poor	**Excellent**
Implementation	Simple	More complex
Search	Fast	**Fast**
Disk utilization	Can decrease	**Better**
Suitable for changing employee count	Moderate	**Excellent**

5. Recommendation

For **5000 employees**, static hashing is sufficient if the employee database is relatively stable.

However, an HR system normally changes over time due to **new employees, resignations, transfers, and record additions**. Therefore, **extendible hashing is the better long-term choice**.

Conclusion

Use **EmployeeID as the hash key** and implement **extendible hashing** with an initial directory of 4 or 8 entries. When a bucket overflows, split the bucket and double the directory when required.

Final choice: Extendible Hashing, because it handles dynamic employee growth efficiently and minimizes long overflow chains.

6. An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.

Combined Index Structure for Airline Reservation System

An airline reservation system needs to support two types of queries:

1. **Point query** – Find a specific flight using FlightNumber.
2. **Range query** – Find flights within a particular date range.

A **combined indexing strategy using B+ Tree indexes** is suitable.

1. Table Structure

```
FLIGHT(
   FlightNumber,
   FlightDate,
   Source,
   Destination,
   DepartureTime,
   AvailableSeats
)
```

2. Index for Point Queries

Create a **B+ Tree index on FlightNumber**.

```
CREATE INDEX idx_flight_number
ON FLIGHT(FlightNumber);
```

Example:

```
SELECT *
FROM FLIGHT
WHERE FlightNumber = 'AI202';
```

The B+ Tree quickly locates the required flight without scanning the complete table.

3. Index for Range Queries

Create another **B+ Tree index on FlightDate**.

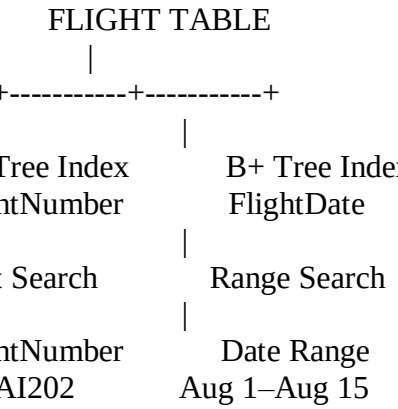
```
CREATE INDEX idx_flight_date
ON FLIGHT(FlightDate);
```

Example:

```
SELECT *
FROM FLIGHT
WHERE FlightDate BETWEEN '2026-08-01' AND '2026-08-15';
```

Because B+ Tree leaf nodes are sorted and linked, the database can efficiently retrieve all flights within the date range.

4. Combined Structure



5. Why Two Indexes?

A single index may not be optimal for both requirements.

Requirement	Recommended Index	Purpose
Specific flight	B+ Tree on FlightNumber	Fast point lookup
Date range	B+ Tree on FlightDate	Efficient range retrieval

6. Optional Composite Index

If the system frequently asks:

"Find flights for a particular flight number within a date range."

Use a composite B+ Tree index:

```
CREATE INDEX idx_flight_number_date
ON FLIGHT(FlightNumber, FlightDate);
```

For example:

```
SELECT *
FROM FLIGHT
WHERE FlightNumber = 'AI202'
AND FlightDate BETWEEN '2026-08-01' AND '2026-08-15';
```

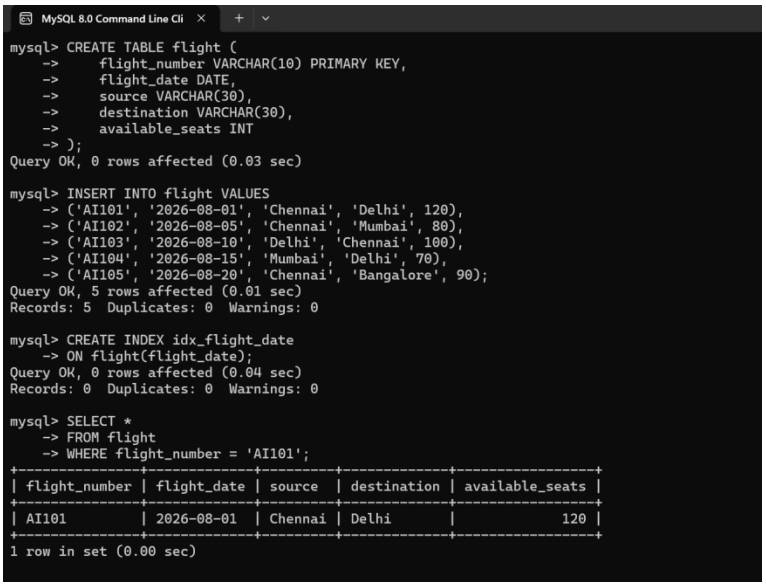
The composite index can efficiently narrow the search by FlightNumber and then by FlightDate.

Conclusion

The best design is to use **two B+ Tree indexes**:

- **B+ Tree on FlightNumber** → fast point queries.
- **B+ Tree on FlightDate** → fast range queries.

If queries commonly combine both attributes, add a **composite (FlightNumber, FlightDate) index**. This provides good performance while keeping the indexing structure practical for a large airline reservation database.



```
mysql> SELECT *
    -> FROM flight
    -> WHERE flight_date BETWEEN '2026-08-01' AND '2026-08-15';
+---------------+-------------+---------+-------------+-----------------+
| flight_number | flight_date | source  | destination | available_seats |
+---------------+-------------+---------+-------------+-----------------+
| AI101         | 2026-08-01  | Chennai | Delhi       |             120 |
| AI102         | 2026-08-05  | Chennai | Mumbai      |              80 |
| AI103         | 2026-08-10  | Delhi   | Chennai     |             100 |
| AI104         | 2026-08-15  | Mumbai  | Delhi       |              70 |
+---------------+-------------+---------+-------------+-----------------+
4 rows in set (0.00 sec)

mysql> EXPLAIN
    -> SELECT *
    -> FROM flight
    -> WHERE flight_date BETWEEN '2026-08-01' AND '2026-08-15';
+----+-------------+--------+------------+-------+-----------------+-----------------+---------+------+------+----------+-----------
-----------+
| id | select_type | table  | partitions | type  | possible_keys   | key             | key_len | ref  | rows | filtered | Extra
          |
+----+-------------+--------+------------+-------+-----------------+-----------------+---------+------+------+----------+-----------
-----------+
|  1 | SIMPLE      | flight | NULL       | range | idx_flight_date | idx_flight_date | 4       | NULL |    4 |   100.00 | Using index
 condition |
+----+-------------+--------+------------+-------+-----------------+-----------------+---------+------+------+----------+-----------
-----------+
1 row in set, 1 warning (0.00 sec)
```

7. For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.

File Organization and Secondary Indexing for University Student Database

A university database may contain thousands of student records and should support fast queries based on **Department** and **CGPA range**.

1. Student Record Structure

STUDENT(
StudentID,
StudentName,
Department,
CGPA,
Year,
Email
)

2. File Organization

Use **heap file organization** for storing the student records.

```
Student Records
      ↓
+--------+--------+--------+--------+
| Block 1| Block 2| Block 3| ...    |
+--------+--------+--------+--------+
```

Heap organization is suitable because students can be inserted, updated, and deleted without maintaining a particular physical order.

However, since queries are frequently performed using Department and CGPA, **secondary indexes** should be created.

3. Secondary Index on Department

Create a **B+ Tree secondary index** on Department.

```
CREATE INDEX idx_student_department
ON STUDENT(Department);
```

Example query:

```
SELECT *
FROM STUDENT
WHERE Department = 'CSE';
```

The index quickly locates the students belonging to the CSE department.

4. Secondary Index on CGPA

Create another **B+ Tree secondary index** on CGPA.

```
CREATE INDEX idx_student_cgpa
ON STUDENT(CGPA);
```

Example:

```
SELECT *
FROM STUDENT
WHERE CGPA BETWEEN 8.0 AND 9.5;
```

Since B+ Tree keys are sorted, the database can efficiently locate the starting CGPA and scan the linked leaf nodes until the range ends.

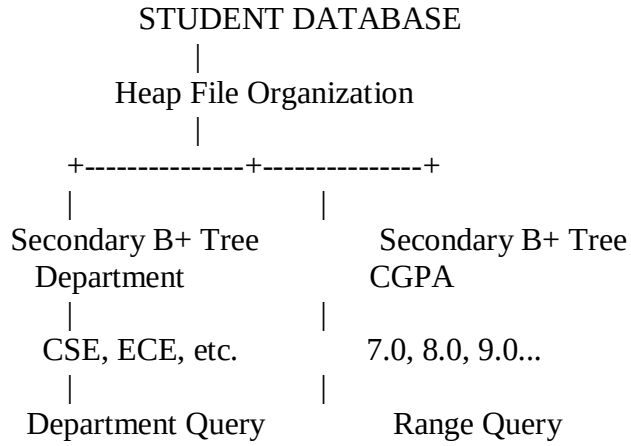
5. Combined Query

For a query such as:

```
SELECT *
FROM STUDENT
WHERE Department = 'CSE'
AND CGPA BETWEEN 8.0 AND 9.5;
```

The DBMS can use the **Department index**, the **CGPA index**, or both depending on the query optimizer and data distribution.

6. Proposed Architecture

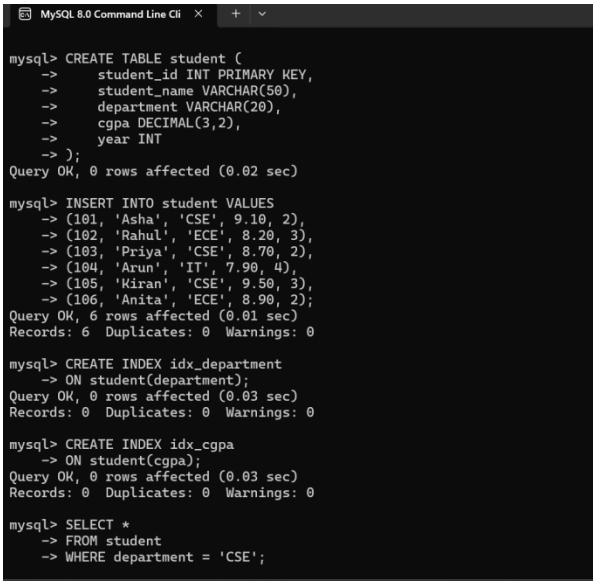


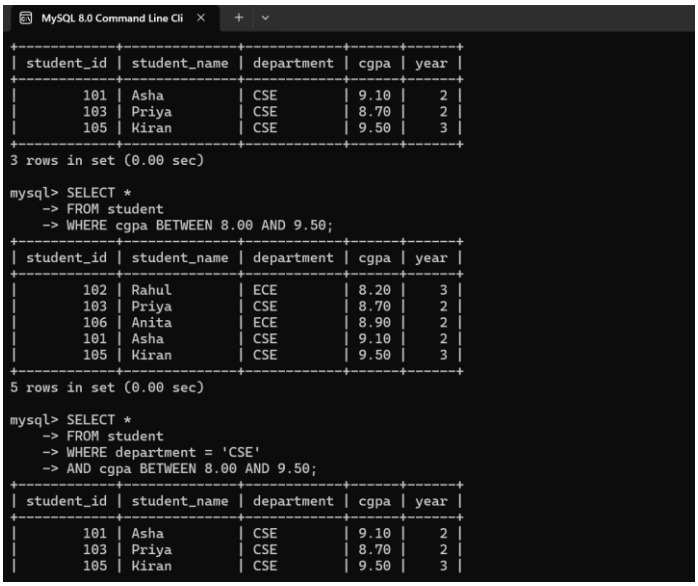
7. Analysis

Requirement	Design Choice	Reason
Store student records	Heap file	Easy insertion/deletion
Search by Department	B+ Tree secondary index	Fast equality search
Search by CGPA range	B+ Tree secondary index	Efficient range search
Department + CGPA query	Both indexes	Reduces records examined
Large student database	Indexed organization	Reduces disk I/O

Conclusion

For the university student database, use **heap file organization** for flexible storage and create **secondary B+ Tree indexes on Department and CGPA**. The Department index provides fast department-based queries, while the CGPA index efficiently handles range queries. This combination reduces disk I/O and provides good performance for large student databases.


MySQL 8.0 Command Line Cli
mysql> CREATE TABLE student (
    ->   student_id INT PRIMARY KEY,
    ->   student_name VARCHAR(50),
    ->   department VARCHAR(20),
    ->   cgpa DECIMAL(3,2),
    ->   year INT
    ->);
Query OK, 0 rows affected (0.02 sec)
mysql> INSERT INTO student VALUES
    -> (101, 'Asha', 'CSE', 9.10, 2),
    -> (102, 'Rahul', 'ECE', 8.20, 3),
    -> (103, 'Priya', 'CSE', 8.70, 2),
    -> (104, 'Arun', 'IT', 7.90, 4),
    -> (105, 'Kiran', 'CSE', 9.50, 3),
    -> (106, 'Anita', 'ECE', 8.90, 2);
Query OK, 6 rows affected (0.01 sec)
Records: 6 Duplicates: 0 Warnings: 0
mysql> CREATE INDEX idx_department
    -> ON student(department);
Query OK, 0 rows affected (0.03 sec)
Records: 0 Duplicates: 0 Warnings: 0
mysql> CREATE INDEX idx_cgpa
    -> ON student(cgpa);
Query OK, 0 rows affected (0.03 sec)
Records: 0 Duplicates: 0 Warnings: 0
mysql> SELECT *
    -> FROM student
    -> WHERE department = 'CSE';


MySQL 8.0 Command Line Cli
+------------+--------------+------------+------+------+
| student_id | student_name | department | cgpa | year |
+------------+--------------+------------+------+------+
101	Asha	CSE	9.10	2
103	Priya	CSE	8.70	2
105	Kiran	CSE	9.50	3
+------------+--------------+------------+------+------+
3 rows in set (0.00 sec)
mysql> SELECT *
    -> FROM student
    -> WHERE cgpa BETWEEN 8.00 AND 9.50;
+------------+--------------+------------+------+------+
| student_id | student_name | department | cgpa | year |
+------------+--------------+------------+------+------+
102	Rahul	ECE	8.20	3
103	Priya	CSE	8.70	2
106	Anita	ECE	8.90	2
101	Asha	CSE	9.10	2
105	Kiran	CSE	9.50	3
+------------+--------------+------------+------+------+
5 rows in set (0.00 sec)
mysql> SELECT *
    -> FROM student
    -> WHERE department = 'CSE'
    -> AND cgpa BETWEEN 8.00 AND 9.50;
+------------+--------------+------------+------+------+
| student_id | student_name | department | cgpa | year |
+------------+--------------+------------+------+------+
101	Asha	CSE	9.10	2
103	Priya	CSE	8.70	2
105	Kiran	CSE	9.50	3

8. Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.

Disk Block Utilization Analysis

Given:

- Number of records = **100,000**
- Record size = **50 bytes**
- Block size = **4 KB = 4096 bytes**

1. Records per Block

Records per block=⌊504096⌋ =81 records/block

The remaining space in each block is:

4096−(81×50)=46 bytes

2. Number of Blocks Required

Blocks=⌈81100000⌉ =1235 blocks

Total actual data size:

100000×50=5,000,000 bytes≈4.77 MB

Total allocated block space:

1235×4096=5,058,560 bytes

Therefore, block utilization is:

5,058,5605,000,000×100 =98.84%

3. Comparison of File Organizations

File Organization	Blocks Required	Utilization	Main Characteristic
Heap	1235	**98.84%**	Records stored in any available space
Sorted	1235	**98.84%**	Records maintained in sorted order
Hashed	1235*	**≈98.84%**	Records distributed using hash function

*Assuming an ideal hash distribution with negligible overflow.

Heap File

Records are inserted wherever free space is available.

- No ordering requirement.
- Very efficient for insertion.
- Same basic block utilization: **98.84%**.

Sorted File

Records are maintained according to a sorting key.

- Same number of records per block.
- Approximately **98.84% utilization**.
- Extra cost may occur during insertion because records may need to be moved.

Hashed File

A hash function distributes records among buckets.

- With an ideal distribution, approximately **98.84% utilization**.
- Overflow buckets can reduce utilization if the distribution is poor.

Final Analysis

All three organizations require approximately **1,235 blocks** because the record size and block size determine the basic storage requirement.

Block Utilization≈98.84%

The main difference is **not the basic storage efficiency**, but how the organizations handle access:

- **Heap:** best for simple and frequent insertions.
- **Sorted:** good for ordered and range-based retrieval.
- **Hashed:** best for fast equality searches.

Thus, with 100,000 records of 50 bytes each, **all three can achieve about 98.84% block utilization under ideal conditions**, while their performance differs based on the type of query.

9. A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.

Composite Indexing Strategy for 500 Million Social Media Posts

A social media platform with **500 million posts** needs fast retrieval based on user_id, timestamp, and hashtag. Since these fields support different query patterns, a combination of **B+ Tree and inverted indexes** is appropriate.

1. Post Table

```
POST(
   post_id,
   user_id,
   timestamp,
   hashtag,
   content
)
```

2. Index on User and Timestamp

Create a composite B+ Tree index:

```
CREATE INDEX idx_user_timestamp
ON POST(user_id, timestamp);
```

This is useful for queries such as:

```
SELECT *
FROM POST
WHERE user_id = 10025
AND timestamp BETWEEN '2026-08-01' AND '2026-08-26';
```

The index first locates the user_id and then efficiently searches the required timestamp range.

3. Index on Hashtag

Hashtags can have many-to-many relationships with posts, so an **inverted index** is suitable.

```
#cricket → Post101, Post245, Post890, ...
#music   → Post120, Post456, Post900, ...
#travel  → Post150, Post700, Post950, ...
```

This allows fast retrieval of all posts containing a particular hashtag.

4. Composite Index on Hashtag and Timestamp

For queries such as:

Find posts containing #cricket during a particular time period.

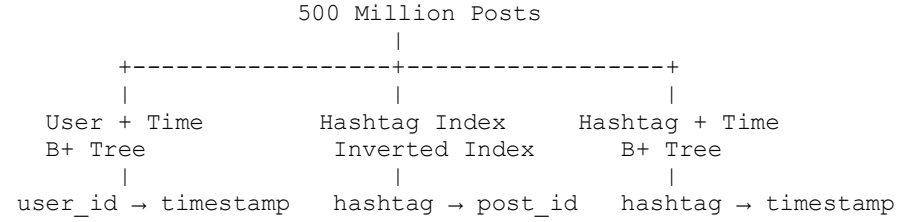
Use:

```
CREATE INDEX idx_hashtag_timestamp
ON POST(hashtag, timestamp);
```

Example:

```
SELECT *
FROM POST
WHERE hashtag = '#cricket'
AND timestamp BETWEEN '2026-08-01' AND '2026-08-26';
```

5. Proposed Structure



6. Why This Strategy?

Query Requirement	**Index**	**Reason**
Posts by user	B+ Tree	Fast equality search
User's posts by time	(user_id, timestamp) B+ Tree	Efficient range query
Posts by hashtag	Inverted index	Very fast hashtag lookup
Hashtag by time	(hashtag, timestamp) B+ Tree/inverted structure	Efficient filtering
500M records	Partitioning + indexes	Reduces search space

7. Partitioning

For **500 million posts**, indexes should be combined with **partitioning**, particularly by time.

For example:

```
Posts
|
+-- 2026-01
```

+-- 2026-02
+-- 2026-03
...
+-- 2026-08

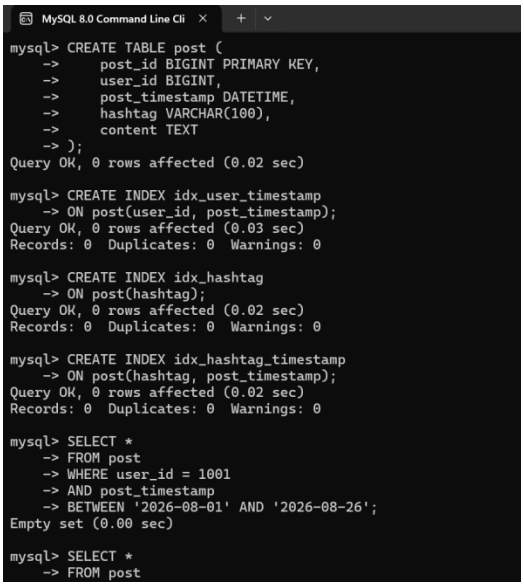
A query for August posts does not need to search older partitions.

Conclusion

The recommended strategy is:

Partition by timestamp + B+ Tree(user_id,timestamp) + Hashtag inverted index

Optionally, maintain a **(hashtag, timestamp) composite index** for frequent hashtag-and-time queries. This design scales much better for **500 million posts** because it reduces both the number of records searched and the indexing overhead.



10. Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.

Performance Comparison of File Organizations

For a supermarket billing system with **1 million daily transactions**, the choice of file organization depends on whether the system prioritizes insertion, deletion, or searching.

Operation	**Heap File**	**Sorted File**	**Hashed File**
Insert	**Very fast – O(1)**	Slow – O(N)	**Fast – O(1) average**
Delete	Moderate – O(N) without index	Moderate – O(N)	**Fast – O(1) average**
Search	Slow – O(N)	**Fast – O(log N)** for ordered key	**Very fast – O(1) average** for equality
Range search	Slow	**Excellent**	Poor
Overflow handling	Not usually required	Reorganization may be needed	Overflow buckets possible
Best use	Heavy insert workload	Ordered/range queries	Exact-key queries

1. Heap File

Transactions are stored in any available space.

New Transaction → First Available Block

Advantages:

- Very fast insertion.
- No sorting overhead.
- Suitable when transactions are continuously generated.

Disadvantage: Searching for a particular transaction without an index may require scanning a large portion of the file.

For **1 million daily transactions**, this makes direct searching expensive.

2. Sorted File

Transactions are maintained in sorted order, for example by Transaction_ID or Transaction_Time.

Advantages:

- Binary search can locate records quickly.
- Excellent for range queries such as:

WHERE Transaction_Time BETWEEN '10:00' AND '11:00'

Disadvantages:

- Inserting a new transaction may require shifting records.
- Maintaining sorted order becomes expensive with **1 million new transactions every day**.

Therefore, it is not ideal for the primary transaction file.

3. Hashed File

Use a transaction key such as Transaction_ID with a hash function:

h(Transaction_ID)=Transaction_IDmodB

where **B** is the number of buckets.

Transaction ID
↓
Hash Function
↓
Bucket
↓
Transaction Record

Advantages:

- Very fast exact searches.
- Fast insertion.
- Fast deletion.
- Suitable for retrieving a transaction by Transaction_ID or Bill_ID.

Disadvantage: Hashing is poor for range queries because records are not stored in sorted order.

Overall Analysis

For a supermarket billing system, **hashed file organization is generally the best choice for transaction lookup**, because the system performs huge numbers of inserts and frequently needs exact transaction/bill searches.

A practical design would be:

Hashed File + Secondary B+ Tree Index on Transaction_Time

This gives:

- **Hashing** → fast insert, delete, and exact transaction search.
- **B+ Tree** → efficient date/time range searches.
- **Scalability** → suitable for 1 million transactions per day.

Final recommendation:

Use **hashed organization as the primary file organization**, with a **B+ Tree secondary index** for time/date-based reporting and range queries.